EXERCÍCIO PRÁTICO

Catálogo de produtos com React

ShopCatalog • Consumo de API REST pública

Objetivo

Desenvolver o frontend de uma aplicação web para consulta de produtos, visualização de detalhes, navegação por categorias, pesquisa e gerenciamento de favoritos. A aplicação deverá consumir uma API REST pública, sem cadastro, chave de acesso ou backend próprio.

> Este exercício é formativo e não vale nota. O foco é praticar organização do código, componentes React, rotas, requisições HTTP, notificações e CSS.

1. API REST utilizada

Utilize a DummyJSON Products API, uma API pública de demonstração que não exige autenticação.

Documentação oficial: https://dummyjson.com/docs/products

URL base: `https://dummyjson.com`

Método	Endpoint	Finalidade
GET	/products	Listar produtos.
GET	/products/:id	Consultar os detalhes de um produto.
GET	/products/categories	Consultar as categorias disponíveis.
GET	/products/category/:category	Listar produtos de uma categoria.
GET	/products/search?q=:termo	Pesquisar produtos por texto.

2. Funcionalidades obrigatórias

RF01 — Página inicial e listagem

• Consultar os produtos por meio do Axios.

• Exibir cards com imagem, título, preço, categoria e avaliação.

• Disponibilizar um botão “Ver detalhes” para cada produto.

• Implementar paginação ou um botão “Carregar mais”.

• Exibir indicador de carregamento, mensagem para resultados vazios e tratamento de erros.

RF02 — Detalhes do produto

• Exibir imagem, título, descrição, categoria, preço, desconto, avaliação e estoque; marca quando disponível.

• Disponibilizar um botão para voltar ao catálogo.

• Permitir adicionar ou remover o produto dos favoritos.

RF03 — Categorias

- Consultar e apresentar as categorias disponíveis na API.
- Ao selecionar uma categoria, exibir apenas os produtos correspondentes.
- Permitir retornar à listagem geral.

RF04 — Pesquisa

- Disponibilizar campo de pesquisa no cabeçalho.
- Utilizar o endpoint de busca da API para pesquisar produtos.
- Tratar pesquisa vazia e ausência de resultados.

RF05 — Favoritos com LocalStorage

- Permitir adicionar e remover produtos dos favoritos.
- Criar uma página para listar os produtos favoritos.
- Persistir os favoritos no `localStorage`, inclusive após atualizar ou reabrir o navegador.
- Utilizar `useState` e `useEffect` para gerenciar e persistir o estado.
- Atualizar a interface imediatamente após incluir ou remover um favorito.

  Não utilizar Context API, Redux, Zustand ou outras bibliotecas de gerenciamento global de estado. O estado poderá ser mantido em um componente comum e passado aos componentes filhos por props.

RF06 — Notificações toast

- Utilizar React Toastify para informar quando um produto for adicionado ou removido dos favoritos.
- Exibir notificações de erro quando uma requisição falhar ou ocorrer um problema inesperado.
- As notificações devem ser breves e não bloquear a navegação.

3. Rotas obrigatórias

Rota	Componente	Responsabilidade
/	HomePage	Listagem geral de produtos.
/produtos/:id	ProductDetailsPage	Detalhes do produto.
/categorias	CategoriesPage	Listagem de categorias.
/categorias/:slug	CategoryProductsPage	Produtos de uma categoria.
/favoritos	FavoritesPage	Listagem de favoritos.
*	NotFoundPage	Página para rota inexistente.

Utilize React Router DOM. O cabeçalho e o rodapé devem ser compartilhados entre as páginas, sem duplicar sua implementação.

4. Tecnologias obrigatórias

- React com componentes funcionais e Hooks.
- React Router DOM para rotas e navegação.
- Axios para todas as requisições HTTP à API.

- React Toastify para notificações.
- CSS puro para estilização dos componentes.
- localStorage para persistência dos favoritos.

Não substitua as respostas da API por listas de produtos ou categorias escritas manualmente no código.

5. Configuração inicial

Crie o projeto com Vite e instale as dependências:

```
npm create vite@latest shop-catalog -- --template react
cd shop-catalog
npm install
npm install axios react-router-dom react-toastify
npm run dev
```

Configure o Axios em `src/services/api.js`:

```
import axios from "axios";

const api = axios.create({
  baseURL: "https://dummyjson.com",
  timeout: 10000,
  headers: {
    "Content-Type": "application/json",
  },
});

export default api;
```

Exemplo de serviço em `src/services/productService.js`:

```
import api from "./api";

export async function getProducts(limit = 12, skip = 0) {
  const response = await api.get("/products", {
    params: { limit, skip },
  });
  return response.data;
}

export async function getProductById(id) {
  const response = await api.get(`/products/${id}`);
  return response.data;
}

export async function getCategories() {
  const response = await api.get("/products/categories");
  return response.data;
}

export async function getProductsByCategory(category) {
  const response = await api.get(
    `/products/category/${encodeURIComponent(category)}`
  );
  return response.data;
}

export async function searchProducts(query) {
  const response = await api.get("/products/search", {
    params: { q: query },
  });
  return response.data;
}
```

Trate os erros nos componentes ou em uma camada apropriada. Não deixe falhas de requisição sem tratamento.

6. Organização sugerida do projeto

```
src/
■■■ components/
■   ■■■ Header.jsx
■   ■■■ Footer.jsx
■   ■■■ ProductCard.jsx
■   ■■■ ProductGrid.jsx
■   ■■■ SearchBar.jsx
■   ■■■ Loading.jsx
■   ■■■ ErrorMessage.jsx
■■■ pages/
■   ■■■ HomePage.jsx
■   ■■■ ProductDetailsPage.jsx
■   ■■■ CategoriesPage.jsx
■   ■■■ CategoryProductsPage.jsx
■   ■■■ FavoritesPage.jsx
■   ■■■ NotFoundPage.jsx
■■■ services/
■   ■■■ api.js
■   ■■■ productService.js
■■■ hooks/
■   ■■■ useFavorites.js
■■■ styles/
■   ■■■ global.css
■   ■■■ Header.css
■   ■■■ ProductCard.css
■   ■■■ Pages.css
■■■ App.jsx
■■■ main.jsx
```

A estrutura é uma sugestão. Você pode propor outra organização, desde que mantenha a separação de responsabilidades e evite concentrar toda a aplicação em um único componente.

7. CSS fornecido

Use os estilos abaixo como base visual. Complete o CSS das demais páginas e componentes, mantendo consistência visual e responsividade.

styles/global.css

```
* {
  box-sizing: border-box;
}

:root {
  font-family: Inter, Arial, sans-serif;
  color: #1f2937;
  background: #f8fafc;
}

body { margin: 0; min-width: 320px; }
button, input, select { font: inherit; }
button { cursor: pointer; }
a { color: inherit; text-decoration: none; }

.container {
  width: min(1200px, 92%);
  margin: 0 auto;
}

.page-title {
  margin: 28px 0 20px;
  font-size: 1.8rem;
}

.page-content {
  min-height: 70vh;
  padding-bottom: 40px;
}

.btn {
  display: inline-block;
  border: 0;
  border-radius: 6px;
  padding: 10px 16px;
  background: #2563eb;
  color: white;
  font-weight: 600;
  transition: background 0.2s;
}

.btn:hover { background: #1d4ed8; }
.btn-secondary { background: #475569; }
.message { padding: 24px; text-align: center; color: #64748b; }
.error-message { color: #b91c1c; }

@media (max-width: 600px) {
  .page-title { font-size: 1.4rem; }
}
```

components/Header.css

```
 .header {
  background: #1e293b;
  color: white;
  padding: 16px 0;
}

.header-content {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 20px;
  flex-wrap: wrap;
}

.logo { font-size: 1.4rem; font-weight: 800; }
.nav-links { display: flex; gap: 20px; align-items: center; }
.nav-links a:hover { color: #93c5fd; }

.search-form { display: flex; gap: 8px; }
.search-form input {
  width: 220px;
  max-width: 100%;
  padding: 10px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
}

@media (max-width: 600px) {
  .header-content { flex-direction: column; align-items: stretch; }
  .nav-links { justify-content: center; flex-wrap: wrap; }
  .search-form input { flex: 1; min-width: 0; }
}
```

components/ProductCard.css

```
 .product-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 20px;
}

.product-card {
  display: flex;
  flex-direction: column;
  overflow: hidden;
  background: white;
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  box-shadow: 0 2px 6px #00000008;
  transition: transform 0.2s, box-shadow 0.2s;
}

.product-card:hover {
  transform: translateY(-3px);
  box-shadow: 0 8px 20px #00000012;
}

.product-image {
  width: 100%;
  height: 190px;
  padding: 12px;
  object-fit: contain;
  background: white;
}

.product-info {
  display: flex;
  flex: 1;
  flex-direction: column;
  align-items: flex-start;
  gap: 10px;
  padding: 16px;
}

.product-title { margin: 0; font-size: 1rem; }
.product-category { color: #64748b; font-size: 0.85rem; }
.product-price { color: #047857; font-size: 1.25rem; font-weight: 800; }
.product-rating { color: #b45309; font-size: 0.9rem; }

.product-actions {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin-top: auto;
  padding-top: 8px;
}

@media (max-width: 900px) {
  .product-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}

@media (max-width: 480px) {
  .product-grid { grid-template-columns: 1fr; }
}
```

8. Configuração das notificações toast

No arquivo `main.jsx`, configure o container global de notificações:

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { ToastContainer } from "react-toastify";

import App from "./App";
import "react-toastify/dist/ReactToastify.css";
import "./styles/global.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
```

```
        <App />
        <ToastContainer
          position="top-right"
          autoClose={3000}
          closeOnClick
          pauseOnHover
        />
      </BrowserRouter>
    </React.StrictMode>
  );
```

Exemplo de notificação de sucesso:

```
import { toast } from "react-toastify";

function handleFavorite() {
  toast.success("Produto adicionado aos favoritos!");
}
```

Para falhas na API, utilize `toast.error(...)` quando apropriado, além de tratar os estados de carregamento e erro da interface.

9. Entregáveis

• Repositório Git com o código-fonte completo.

• README.md com instruções de instalação e execução.

• Aplicação executável localmente, sem backend próprio.

• Breve descrição das decisões de organização e das funcionalidades implementadas.

10. Orientações finais

• As telas principais devem carregar dados reais da API, e não listas estáticas no código.

• As rotas devem funcionar também quando uma URL interna for atualizada diretamente no navegador.

• Trate carregamento, resultados vazios e erros HTTP.

• Mantenha páginas, componentes e serviços separados.

• A interface deve funcionar em desktop e dispositivos móveis.

Prazo sugerido: 3 a 5 dias.

> Este exercício não vale nota. Utilize-o como oportunidade para praticar, experimentar soluções e consolidar os conceitos estudados.